Algorithms for Flow Time Scheduling (Thesis Proposal)

Nikhil Bansal

Abstract

In this thesis proposal we address problems in scheduling related to minimizing flow time. The flow time of a job is the time it spends until its service requirement is met. We study three fundamental problems: First, we consider the problem of minimizing weighted flow time on a single machine. We give the first truly online algorithm with a non-trivial competitive ratio. Second, we study online algorithms for minimizing the L_p norms of flow time and stretch (stretch of job is its flow time divided by its size). Both these problems are also studied in the non-clairvoyant setting where the size of a job is not known upon its arrival. We give a general technique to reduce a non-clairvoyant problem to a (usually simpler) clairvoyant problem. Third, is the offline problem of minimizing the total flow time on multiple machines. We give a Quasi-polynomial time approximation scheme (a $1+\epsilon$ approximation algorithm with a running time $n^{O(\log n)}$) for a constant number of machines. The previous best known approximation result for the problem was an $O(\log n)$ approximation, where n is the number of jobs [30, 3].

In this proposal we describe our work on the above mentioned problems. Additionally, we consider some other problems which, while interesting by themselves, also serve to illustrate the applicability of our techniques.

1 Introduction

Scheduling can be described as allocating resources over time. The resources are modeled as machines and the request for resources are modeled as jobs with a service requirement. Given this broad definition, scheduling has been studied in various scenarios such as practical computer systems, production planning in factories, load balancing and so on. Scheduling problems constitute a substantial part of optimization, and have been studied by researchers in various communities such as operations research, algorithms and queueing theory.

Algorithms for scheduling have been studied since the fifties, and their study has led to various ideas and paradigms in the field of algorithms. For example, the earliest online and approximation algorithms known were for the problem of makespan minimization on multiple machines. Again, due to its wide scope, scheduling problems have been studied in various models such has single machine, multiple machine, flow shops, job shops, open shops, jobs with dependencies and precedence constraints and for various measures such as completion time, makespan, flow time, their weighted versions etc.

In the last ten years or so, a major focus has been on algorithms for minimizing flow time. The flow time of a job is defined as the time spend by a job until it meets its service requirement. Flow time closely relates to the user experience as it measures the amount a user

has to wait to get his jobs serviced. Indeed flow time has been considered widely in the context of operating systems, web servers, query time in databases, providing Quality of Service (QoS) guarantees in networks and so on. In this thesis we study some fundamental problems related to minimizing flow time and its variants. We begin by describing the model formally.

1.1 Model and Preliminaries

An instance of a scheduling problem consists of a collection of jobs which arrive dynamically over time. The time when a job J_j arrives is its release time and denoted by r_j. Each job has a service requirement, also known as its size and is denoted by p_j. In the typical scheduling model, it is assumed that p_j becomes known to the scheduling algorithm when J_j arrives[1]. The number of jobs is always denoted by n, and the number of machines is always m. A schedule for a problem instance species for each job various intervals of time where it is executed. Figure 1 shows a possible schedule for $n = 3$ jobs and $m = 1$ machine. A scheduling algorithm is a procedure to construct a schedule from an input instance. For example, some well known scheduling algorithms are:

1. Round Robin (RR): At any time, share the processor equally among all the jobs present at that time.
2. Shortest Job First (SJF): At any time, work on the job which has the shortest size (p_i).
3. Shortest Remaining Time First (SRPT): At any time, work on the job which has the least remaining processing requirement at that time.
4. Shortest Elapsed Time First (SETF): At any time, work on the job which has received the least amount of service at that time.

1.1.1 Objective Functions

Given a schedule, the completion time c_j for a job is the time at which it completes its service requirement. The flow time f_j of a job is time a job spends in the system. i.e. it is the difference between the completion time and the release time ($f_j = c_j - r_j$). Figure 1 shows the flow times of jobs J_1 and J_3. It is a folklore (and easy to prove) result that the scheduling algorithm SRPT is optimal for minimizing total flow time on a single machine.

Another measure closely related to flow time is *stretch*. Given a schedule, the stretch s_j of a job is the ratio of its flow time and its size i.e. $s_j = f_j/p_j$. Stretch can be thought of as *normalized* flow time, and relates the waiting time of the users to their demands. In particular, it models the fact that users are usually willing to wait longer for large jobs as opposed to short jobs. It is unknown whether minimizing the total stretch on a single machine is NP-Hard. Recently it was shown that the algorithm SRPT is 2 competitive for minimizing total stretch on a single machine [32].

[1]Later, we will consider the model where p_j may be unknown to the scheduling algorithm (i.e. p_j remains unknown to the scheduler until J_j meets its service requirement and finishes execution)

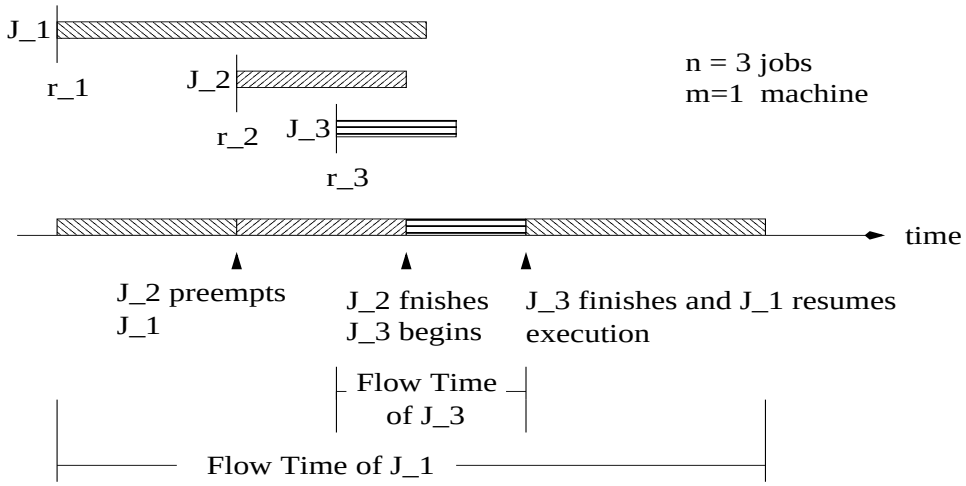


Figure 1: An example of a schedule

1.1.2 Preemption

A schedule is non-preemptive if a job cannot be interrupted once it is started. In a preemptive schedule a job can be interrupted arbitrarily and the execution can be resumed from the point of interruption without any penalty (for example in Figure 1, the job J_2 preempts J_1).

Being unable to preempt places a serious restriction on the quality of schedules with respect to the flow time and stretch metrics, and hence usually flow time and stretch related objectives functions are studied in the preemptive model. In the thesis, we focus only on preemptive scheduling. Note that all the algorithms mentioned above (i.e. RR, SJF, SRPT and SETF) are preemptive.

1.1.3 Non-Clairvoyant Scheduling and Resource Augmentation

In many practical systems, such as in operating systems, apriori information about the size of a job might be hard or impossible to obtain. Motivated by this scenario, Motwani et al [31] introduced the *non-clairvoyant* scheduling model. Here, jobs arrive over time and the size of a job is unknown to the scheduler until the job finishes. A non-clairvoyant algorithm thus schedules the jobs without the knowledge of their sizes.

The competitive ratio of a non-clairvoyant algorithm A is the supremum over all inputs I of n jobs of the ratio $A(I)/Opt(I)$, where $A(I)$ is the cost of A on I and $Opt(I)$ is the cost of the optimal *clairvoyant* algorithm on I. As the comparison is made against a clairvoyant algorithm, usually no non-clairvoyant algorithm can perform too well in this model. For example, it is known that any deterministic (resp. randomized) non-clairvoyant algorithm is $\Omega(n^{1/3})$ (resp $\Omega(\log n)$) competitive for total flow time [31].

Kalyanasundaram and Pruhs [25] proposed the model of *resource augmentation*, where the online algorithm is again compared with an optimal offline clairvoyant adversary but one that has a slower processor. In this model [25], an algorithm is $s-$speed, $c-$competitive if it uses an

s times faster processor than the optimum algorithm and produces a solution that is no more than c times worse. Resource augmentation often provides a nice framework for comparing the performance of algorithms, where the traditional competitive analysis gives a very pessimistic guarantee for all algorithms.

1.1.4 Terminology for Multiple Machines

In a multiple machine setting, we consider two models. The *identical* machines case where a job has the same size on any machine. A more general model is that of *unrelated* machines, where the size of a job J_j is machine dependent, and is described by an arbitrary vector $\vec{p_j} = (p_{1,j}, p_{2,j}, \ldots, p_{m,j})$, where $p_{i,j}$ denotes the size of j machine i.

Another issue while scheduling jobs on multiple machines is that of migration. A schedule is *migratory* if it can move partially executed jobs from one machine to another. Migration of jobs is usually considered unattractive as it incurs a huge overhead (cost) in practice. Throughout this thesis we only consider non-migratory schedules.

1.1.5 Local-Competitiveness

Local-competitiveness is a general technique for analyzing flow time related measures. It has been used frequently in both previous work and our work.

Consider the example of total weighted flow time, it is expressed $\sum_j w_j f_j$, i.e. the sum over all jobs of the weight of job J_j times the flow time of job J_j. Alternatively, we can think of each job of weight w contributing w every unit of time it spends in the system. Then, the total weighted flow time under an algorithm $\mathcal{A}$ can be expressed as $\sum_t w_{\mathcal{A}}(t)$, where $w_{\mathcal{A}}(t)$ is the total weight of jobs alive under $\mathcal{A}$ at time t. Thus, the idea of local-competitiveness is that, to prove c competitiveness of $\mathcal{A}$ it suffices to show that at all times t, $w_{\mathcal{A}}(t) \leq c w_O(t)$, where $w_{\mathcal{O}}(t)$ denotes the total weight possible under any other algorithm $\mathcal{O}$ at time t.

2 Summaries

In this section we outline some problems that we have previously considered and summarize our results for these. For clarity, we also state the related previous work and the open problems with each of these problems.

2.1 Minimizing Weighted Flow Time

In many settings jobs have varying degrees of importance and this is usually represented by assigning weights to jobs. This motivates the following problem.

2.1.1 Problem Statement

We are given a single machine with preemptions. Jobs arrive online and have arbitrary release times, size and weights. The goal is to schedule these to minimize the total weighted flow time.

For general weights the problem is NP-Hard [28]. We will consider the problem in both the clairvoyant and non-clairvoyant models. In the non-clairvoyant model, only the weight of the job is revealed, and its size is unknown until the job completes.

2.1.2 Previous Work

In the clairvoyant model, two results were known prior to our work.

1. An $O(\log^2 P)$ competitive semi-online algorithm [16]. The algorithm is semi-online in the sense that it requires the knowledge of P in advance. Here P denotes the ratio of the maximum to minimum job size.

2. A $(1+\epsilon)$-speed, $(1+1/\epsilon)$ competitive algorithm [11].

In the non-clairvoyant model, no previous results are known for weighted flow time. The only related result is a $1+\epsilon$-speed, $1+1/\epsilon$-competitive algorithm for *unweighted* flow time [25].

2.1.3 Our Results

1. We give an algorithm that is k-competitive if the job weights can have k different values. Since jobs can be rounded to powers of 2, this directly gives an $O(\log W)$ competitive algorithm, where W denotes the ratio of the maximum to minimum weight.

   Our algorithm yields the first $O(1)$-competitive algorithm if $k = O(1)$. The algorithm in [16] was $\Omega(\log P)$ competitive even for $k = 2$.

   Our algorithm is also truly online, which answers an open question of [16].

2. We also give an $O(\log n + \log P)$ competitive semi-online algorithm. The algorithm is semi-online in the sense that it requires the knowledge of n,P and W in advance. Thus our algorithm yields a better guarantee than that of [16], unless $P = 2^{o(\sqrt{\log n})}$.

3. We give a $(1+\epsilon)$-speed $(1+\frac{1}{\epsilon})$-competitive algorithm for minimizing the total weighted flow time in the *non-clairvoyant* setting.

The proofs of all these results rely on local-competitiveness, however they are rather technical and are not included in this proposal. The details can be found in [6].

2.1.4 Open Problems

1. Obtaining an $O(1)$ competitive online algorithm, or even an $O(1)$ approximation algorithm for weighted flow time is a major open problem. I would really like to solve this problem for my thesis.

2. Nothing is known for minimizing weighted flow time non-clairvoyantly *without* resource augmentation. We believe that an $O(\log W \log n)$ competitive algorithm can be obtained for this case by combining our techniques with that for an $O(\log n)$ competitive randomized non-clairvoyant algorithm for unweighted flow time [24, 10].

2.2 Stretch Scheduling in the Non-clairvoyant Model

It is known that the algorithm Shortest Remaining Processing Time (SRPT) is 2-competitive for minimizing stretch on a single machine [32]. In this thesis we study the problem in the non-clairvoyant model. The problem was posed in [10]. It is also interesting theoretically as local-competitiveness is provable inapplicable to this problem. We solve this using a new technique described in Section 3.

Minimizing total stretch can be thought of as a special case of minimizing total weighted flow time, where the weight of a job is inversely proportional to its size. However, what makes the problem interesting and considerably harder in a non-clairvoyant setting is that the size es (hence weights) are not known. Hence not only does the online scheduler have no idea of job sizes, it also has no idea as to which job is more important (has a higher weight).

An interesting aspect of slowdown is that neither resource augmentation nor randomization seems to help in the non-clairvoyant case. This is in sharp contrast to total flow time where a $(1+\epsilon)$ speed gives a $(1+\frac{1}{\epsilon})$ competitive algorithm [25], and randomization allows us to obtain an $O(\log n)$ competitive non-clairvoyant algorithm [24, 10].

To get a feel for the problem, suppose there are n jobs of size $1, 2, 4, \ldots, 2^{n-1}$ all of which are released at time $t = 0$. By scheduling these in the order of increasing job sizes, the optimal clairvoyant algorithm has a total stretch of $O(n)$. However, the non-clairvoyant has no way of figuring out the size 1 job until it has spent at least 1 unit of time on all the size 1 jobs (or on about half the jobs if the algorithm is randomized). Thus the size 1 job will have a stretch of $\Omega(n)$. Arguing similarly for other jobs, implies that the total stretch is $\Omega(n^2)$, hence giving a lower bound of $\Omega(n)$ on the competitive ratio. Moreover, since all jobs arrive at $t = 0$, a speedup of k implies that the flow time reduces by k times and hence with an $O(1)$ speedup the competitive ratio is still $\Omega(n)$.

2.2.1 Our Results

We have the following results [7]:

1. Without resource augmentation, any randomized online algorithm for total stretch is $\Omega(\min(n, P))$ competitive. Even with an $O(1)$ speedup any randomized online algorithm is $\Omega(\min(n, \log P))$ competitive.

2. We give an $O(1)$-speed, $O(\log^2 P)$-competitive algorithm. This result is however subsumed a more general result described below in section 2.3.

3. For the case when all jobs arrive at time $t = 0$, we show that any randomized algorithm is $\Omega(\min(n, \log P))$ competitive and give a tight $O(\min(\log P, n))$ competitive algorithm.

2.2.2 Open Problems

It is open if the gap between the lower bound of $\Omega(\log P)$ and the upper bound of $O(\log^2 P)$ for the general problem can be closed.

2.3 Scheduling in the L_p Norm

It is well known that Shortest Remaining Processing Time (SRPT) is optimal for minimizing the *total* flow time. Similarly, First Come First Serve (FCFS) is optimal of minimizing the *maximum* flow time. However, each of the algorithms has its drawbacks. A commonly cited reason against using SRPT in a real system is the fear that it may "starve" large jobs (i.e. large jobs might have a extremely high flow time or stretch). Similarly, FCFS might have a very poor average behavior (for example, consider $n-1$ small jobs stuck behind a large job).

To strike a compromise between the sum and the max metric we propose the L_p norms of flow time and stretch as a measure of fairness. While L_p norms of measures such as completion time and load have been considered previously [18, 1, 2], no previous work has been done on flow time related measures.

2.3.1 Results

Table 2.3.1 below summarizes our results: The main contribution is the following.

1. We show that any online randomized algorithm for minimizing the L_p norm of flow time or stretch is $n^{\Omega(1)}$ competitive for $p > 1$. The result is perhaps surprising since there are optimum online algorithms for $p = 1$ and $p = \infty$.

2. Given the strong lower bounds, we use resource augmentation. We show that the algorithms Shortest Job First (SJF) and SRPT are close to optimal in the clairvoyant setting both with respect to the L_p norms of flow time and the L_p norms of stretch. We also give a close to optimal strategy Shortest Elapsed Time First (SETF) for the non-clairvoyant setting.

3. We show that resource augmentation does not make the problem trivial. In fact, the algorithm Round Robin (RR) which is widely considered as a fair algorithm is $(1+\epsilon)$-speed $\Omega(n^{1-2\epsilon p})$ competitive. Hence for sufficiently small ϵ it is not $O(1)$ competitive.

Algorithm	Speed	Competitive Ratio for L_p Norm of Flow	Competitive Ratio for L_p Norm of Stretch
General Clairvoyant Algorithm	1	$\omega(1)$	$\omega(1)$
SJF	$(1+\epsilon)$	$O(1/\epsilon)$	$O(1/\epsilon)$
SRPT	$(1+\epsilon)$	$O(1/\epsilon)$	$O(1/\epsilon)$
SETF	$(1+\epsilon)$	$O(1/\epsilon^{2+2/p})$	$O(\frac{1}{\epsilon^{3+1/p}} \cdot \lg^{1+1/p} P)$
RR	$(1+\epsilon)$	$\Omega(n^{1-2\epsilon p})$	$\Omega(n)$
General Nonclairvoyant Algorithm	$(1+\epsilon)$		$\Omega(\min\{n, \log P\})$

Table 1: Resource augmentation results for the standard average QoS algorithms

The main idea of the analysis is to think of the pth power of flow time of a job J_j as $\sum_{t=0}^{\infty} age(j,t)^{p-1}$, where $age(j,t)$ is the age (time since the job arrived) of J_j at time t, if

$r_j \leq t \leq c_j$ and 0 otherwise. We then use local-competitiveness, in particular, we show that at all times the total sum of $p-1^{th}$ powers of ages of all jobs is no more than a constant times that under any optimal algorithm. The technical details are rather involved and can be found in [8].

For the non-clairvoyant results, we use the technique described in Section 3.

2.4 Approximation Schemes for flow time on multiple machines

We now consider two problems related to minimizing flow time on multiple machines in the offline setting.

Our first problem considers minimizing the total flow time. The best known algorithm is an $O(\log(\min\{\frac{n}{m}, P\}))$ competitive online algorithm for identical parallel machines. The algorithm involves migration of jobs. An equally effective algorithm in terms of competitive ratio but which does not require job migration is also known [3]. Obtaining a constant factor approximation or a PTAS is considered one of the major open problems in scheduling [34, 30, 3]. We make some progress on the problem by giving a Quasi-polynomial time approximation scheme (QPTAS), i.e. an algorithm which produces an $1+\epsilon$ approximate solution and has running time $O(n^{\log^{O(1)} n})$, for a constant number of machines. Our result uses a new technique where we show how to enumerate all possible schedules under SRPT using quasi-polynomial space. This technique is described in Section 4 and is also used to obtain a QPTAS for a problem discussed in Section 2.5 below.

Second, we consider the problem of minimizing the maximum flow time on m machines. Note that if all the jobs are released at time 0, this reduces to the classical problem of makespan minimization, for which a lot of previous work has been done [22, 21, 29, 23]. For general release times the only result known is a $3-2/m$ online algorithm [12]. Obtaining a PTAS was open for this problem [12, 34]. We partially solve the problem by obtaining a PTAS for constant number of machines.

2.4.1 Results

1. Our main result is an algorithm for minimizing total flow time on identical machines, which produces a $1+\epsilon$ approximate solution and has running time $n^{O(m \log n/\epsilon^2)}$. Thus for a fixed number of machines, this gives a QPTAS.

2. The above result easily extends to the case when the machines are unrelated, and additionally, even if our measure is weighted flow time, but we have that either W or P is poly-bounded in n. In this case our approximation scheme has running time $n^{O(m \log^2 n/\epsilon^3)}$.

   For weighted flow time, the only previous approximation scheme known holds only for a single machine and has a running time $n^{O(\log W \log P/\epsilon^3)}$ [15]. For the case where either P or W is poly-bounded in n, their result can be strengthened to give an approximation scheme with running time $n^{O(\log^2 n/\epsilon^3)}$. Thus our result for weighted flow time can be thought of as extending the result of [15] to multiple machines. However, in a way the result of [15] is more general than ours: Our result does not hold if both W and P are arbitrary.

3. For maximum flow time on unrelated machines, we give an algorithm that produces a $1+\epsilon$ approximate solution in time $O(n^{m/\epsilon})$.

2.4.2 Open Problems

1. Obtaining a constant approximation or a PTAS for total unweighted flow time is the major open problem.

2. A PTAS for weighted flow time on a constant number of machines would be a significant breakthrough. An interesting intermediate goal might be to extend the result of [15] to multiple machines with arbitrary P and W (i.e. obtain an approximation scheme with running time $O(n^{poly(\log W, \log P, 1/\epsilon, m)})$ when both W and P are arbitrary).

3. For the maximum flow time problem we do not know if the dependence of the number of machines can be made polynomial.

4. An interesting problem is to obtain a PTAS for minimizing stretch on multiple machines. At present, a PTAS is known for the single machine case [15, 13] and an O(1) approximation for the multiple machines case [32].

5. It would also be interesting to see if we could apply these techniques to obtain approximation schemes for the L_p norms of flow time and stretch. There has been no work of this problem so far.

2.5 Combining Flow time Scheduling and Call Admission

We study the problem of scheduling on a single machine to minimize flow time (also waiting time, defined as $f_j - p_j$) when jobs can be rejected at some cost. Each job J_j has a release date r_j, a processing time p_j, and we may at any time cancel a job at some cost.

Admission control has been studied for a long time in circuit routing problems (see for e.g. [14]). However, the focus there is on maximizing the throughput of the network. In scheduling problems, the model of rejection with penalty was first considered in the context of minimizing makespan on multiple machines [9], they gave an $O(1)$ approximation. Minimizing weighted *completion* time with rejections offline has also been studied [17]

2.5.1 Our Results

1. For the case when all jobs have an equal rejection penalty we give a 2-competitive online algorithm for flow-time (resp. waiting time).

   For the offline version, we give a quasi-polynomial time approximation scheme.

2. We extend the above result to weighted flow time with rejections, where each job has a weight associated with it. We consider two models for penalty: first, where every job has the same penalty c of rejection, and second, where penalty of a job is proportional to its weight (i.e. a job of weight w has penalty wc). Note that in both the variants, by choosing c very large, we obtain classic weighted flow time minimization problem as

a special case. We give an $O(\log^2 W)$ competitive algorithm for both the variants, using the ideas from [6].

3. Finally, we consider the most general problem of minimizing the weighted flow time with arbitrary penalties. We show rather strong lower bounds for the case of arbitrary penalties, even for the case of unweighted jobs.

   Motivated by the lower bounds, we use resource augmentation and give a $1+\epsilon$-speed, $O(\frac{1}{\epsilon}(\log W + \log C)^2)$-competitive online algorithm for minimizing weighted flow time with arbitrary penalties on a $(1+\epsilon)$ speed processor. Here C is the ratio between the maximum to minimum penalty.

The proofs for competitiveness of the online algorithms rely on local-competitiveness (with a different idea to account for the rejection cost). The offline result is proved using the technique described in Section 4.

3 A General Technique for Non-Clairvoyant Algorithms

In this section, we give a technique for analyzing non-clairvoyant algorithms in cases where local-competitiveness does not work. In this section, we give a $(1+\epsilon)$-speed $O(1/\epsilon^{2+2/p})$-competitive algorithm for minimizing the L_p norm of flow time non-clairvoyantly, and a $(1+\epsilon)$-speed $(1/\epsilon \cdot \log P)^{O(1)}$-competitive algorithm for minimizing the L_p norm of stretch non-clairvoyantly [8].

3.1 Flow Analysis

We will consider the Shortest Elapsed Time First *SETF*, which at any time works on the job which has received the least amount of service. If there are multiple jobs with the same amount of service, it time-shares among these jobs.

We first reduce the analysis of *SETF* to that of *MLF*, Our *MLF* is a variation of the standard Multilevel Feedback Queue algorithm where the quanta for the queues are set as a function of ϵ. Let $\ell_i = \epsilon((1+\epsilon)^i - 1)$, $i \geq 0$, and let $q_i = \ell_{i+1} - \ell_i = \epsilon^2(1+\epsilon)^i$, $i \geq i$. In *MLF* a job J_j is in *level k at time t* if the work done on J_j by time t is $\geq \ell_k$ and $< \ell_{k+1}$. *MLF* maintains the invariant that it is always running the earliest arriving job in the smallest nonempty level.

We begin with some notation. For an algorithm A and instance $\mathcal{I}$, $F^p(A(\mathcal{I}), s)$ (resp. $S^p(A(\mathcal{I}), s)$) denotes the pth power of flow time (resp stretch) when A is run on instance $\mathcal{I}$ using an s speed processor.

Lemma 1 *For any instance $\mathcal{I}$, and for any $J_j \in \mathcal{I}$, J_j completes in SETF$(\mathcal{I}, 1+\epsilon)$ before J_j completes in MLF$(\mathcal{I}, 1)$. In particular, this implies that*

$$F^p(SETF(\mathcal{I}), s) \leq F^p(MLF(\mathcal{I}), s/(1+\epsilon)) \quad \textit{and} \quad S^p(SETF(\mathcal{I}), s) \leq S^p(MLF(\mathcal{I}), s/(1+\epsilon)) \tag{1}$$

Now our goal will be to relate *MLF* and *Opt*. Let the original instance be $\mathcal{I}$. Let $\mathcal{J}$ be the instance obtained from $\mathcal{I}$ as follows. Consider a job J_j and let i be the smallest integer such that $p_j + \epsilon \leq \epsilon(1+\epsilon)^i$. The processing time of J_j in $\mathcal{J}$ is then $\epsilon(1+\epsilon)^i$. Let $\mathcal{K}$ be the instance obtained from $\mathcal{J}$ by decreasing each job size by ϵ. Thus, each job in $\mathcal{K}$ has size $\ell_k = \epsilon((1+\epsilon)^k - 1)$ for some k. Note that in this transformation from $\mathcal{I}$ to $\mathcal{K}$, the size of a job doesn't decrease, and it increases by at most a factor of $(1+\epsilon)^2$. Since *MLF* is has the property that increasing the length of a particular job will not decrease the completion time of any job, we can conclude that

$$F^p(MLF(\mathcal{I}), s/(1+\epsilon)) \leq F^p(MLF(\mathcal{K}), s/(1+\epsilon)) \tag{2}$$

Finally we create an instance $\mathcal{L}$ by replacing each job of size $\epsilon((1+\epsilon)^k - 1)$ job in $\mathcal{K}$ by k jobs of size $q_1, \ldots, q_{k-1}$. Note that $\ell_k = q_0 + q_1 + \ldots, q_{k-1}$. For a job of $J_j \in \mathcal{K}$, we denote the corresponding jobs in $\mathcal{L}$ by $J_{j_0}, J_{j_1}, \ldots, J_{j_{k-1}}$.

Notice that any time t, $SJF(\mathcal{L})$ is working on a job $J_{j_b} \in \mathcal{L}$ if and only if $MLF(\mathcal{K})$ is working on job $J_j \in \mathcal{K}$ that is in level b at time t. In particular, this implies that the completion time of J_j in $MLF(\mathcal{K})$ is exactly the completion time of some job $J_{j_b} \in SJF(\mathcal{L})$. Hence,

$$F^p(MLF(\mathcal{K}), s/(1+\epsilon)) \leq F^p(SJF(\mathcal{L}), s/(1+\epsilon)) \tag{3}$$

Since *SJF* is $1+\epsilon$-speed, $O(1+1/\epsilon)$-competitive for L_p norm of flowtime [8], we know that

$$F^p(SJF(\mathcal{L}), s/(1+\epsilon) = O(1/\epsilon^p) F^p(Opt(\mathcal{L}), s/(1+\epsilon)^2) \tag{4}$$

We relate the optimal schedule for $\mathcal{L}$ back to the optimal schedule for $\mathcal{I}$. To do this we first relate $\mathcal{L}$ to $\mathcal{J}$ as follows. Let $\mathcal{L}(k)$ denote the instance obtained from $\mathcal{J}$ by multiplying each job size in $\mathcal{J}$ by $\epsilon/(1+\epsilon)^k$. Next, we remove from $\mathcal{L}(k)$ any job whose size is less than ϵ^2. We claim that $\mathcal{L} = \mathcal{L}(1) \cup \mathcal{L}(2) \cup \ldots$. To see this, let us consider some job $J_j \in \mathcal{J}$ of size $\epsilon(1+\epsilon)^i$. Then, $L(1)$ contains the corresponding job $J_{j_{i-1}}$ of size $\epsilon/(1+\epsilon) \cdot \epsilon(1+\epsilon)^i = \epsilon^2(1+\epsilon)^{i-1} = q_{i-1}$. Similarly $\mathcal{L}(2)$ contains the job $J_{j_{i-2}}$ of size q_{i-2} and so on. Thus, $\mathcal{L}$ is exactly $\mathcal{L}(1) \cup \mathcal{L}(2) \cup \ldots$. Summarizing, we have that the $\mathcal{L}(k)'s$ are geometrically scaled down copies of J and that $\mathcal{L}$ is exactly the union of these $\mathcal{L}(k)'s$.

Our idea at a high level is as follows: Given a *good* schedule of $\mathcal{J}$, we obtain a good schedule for $\mathcal{L}(k)$. This will be easy to do as $\mathcal{L}(k)$ is a scaled down version of $\mathcal{J}$. Finally, we will superimpose the schedules for each $\mathcal{L}(k)$ to obtain a schedule for $\mathcal{L}$. This will give us a procedure to obtain a *good* schedule for $\mathcal{L}$ given a good schedule for $\mathcal{J}$. To put everything in place, we need the following lemma from [8] while relates $\mathcal{J}$ and $\mathcal{L}(k)$.

Lemma 2 *Let $\mathcal{L}(k)$ be as defined above. Let $F(s, J_i, \mathcal{G}))$ (respectively $S(s, J_i, \mathcal{G}))$ denote the flow time (respectively stretch) of job $J_i \in \mathcal{G}$ when SJF is run on $\mathcal{G}$ with a speed s processor. Then, for all $x \geq 1$,*

$$F(\epsilon(1+\epsilon)^{-k} \cdot x \cdot s, J_{i_k}, \mathcal{L}(k)) \leq \frac{1}{x} F(s, J_i, \mathcal{J})$$

$$S(\epsilon(1+\epsilon)^{-k} \cdot x \cdot s, J_{i_k}, \mathcal{L}(k) \leq \frac{(1+\epsilon)^k}{\epsilon x} S(s, J_i, \mathcal{J})$$

Lemma 3 $F^p(Opt(\mathcal{L}), 1+2\epsilon) = O(1/\epsilon^2)F^p(SJF(\mathcal{J}), 1)$

Proof: Given the schedule $SJF(\mathcal{J})$, we construct a following schedule A for $\mathcal{L}$ as follows. The jobs in $\mathcal{L}(k)$ are run with a speed $x_k = \epsilon(1+\frac{\epsilon}{1+\epsilon})^{-k}$ processor using the algorithm SJF. Note that the total speed required by A is at most

$$\sum_{i=1}^{\infty} x_i = \sum_{i=1}^{\infty} \epsilon \frac{1}{(1+\frac{\epsilon}{1+\epsilon})^i} = \frac{\epsilon}{1-\frac{1+\epsilon}{1+2\epsilon}} = 1+2\epsilon$$

By Lemma 2, $F^p(A(\mathcal{L}(k)), 1+2\epsilon)$ will be at most $\left(\frac{\epsilon(1+\epsilon)^{-k}}{x_k}\right)^p = \left(\frac{(1+2\epsilon)}{(1+\epsilon)^2}\right)^{kp}$ times $F^p(SJF(\mathcal{J}, 1)$. Hence,

$$\frac{F^p(A(\mathcal{L}), 1+2\epsilon)}{F^p(SJF(\mathcal{J}), 1)} \leq \sum_{i=1}^{\infty} \left(\frac{1+2\epsilon}{(1+\epsilon)^2}\right)^{ip} \sum_{i=0}^{\infty} \left(1 - \frac{\epsilon^2}{(1+\epsilon)^2}\right)^i = O(1/\epsilon^2)$$

The proof then follows because Opt is at least as good as A. □

Hence by Lemma 3, $1+\epsilon$-speed, $O(1/\epsilon)$-competitiveness of SJF for L_p norm of flow time, and that the jobs lengths in $\mathcal{J}$ are at most $(1+\epsilon)^2$ times as long as they are in $\mathcal{I}$, we get that

$$\begin{aligned} F^p(Opt(\mathcal{L}), s/(1+\epsilon)^2) &= O(1/\epsilon^2) \cdot F^p(SJF(\mathcal{J}), \frac{s}{(1+2\epsilon)(1+\epsilon)^2}) & (5) \\ &= O(1/\epsilon^{p+2}) \cdot F^p(Opt(\mathcal{J}), \frac{s}{(1+2\epsilon)(1+\epsilon)^3}) & (6) \\ &= O(1/\epsilon^{p+2}) \cdot F^p(Opt(\mathcal{I}), \frac{s}{(1+2\epsilon)(1+\epsilon)^5}) & (7) \end{aligned}$$

By stringing together the inequalities 1, 2, 3, 4, 5, 6, 7, we find that

$$F^p(SETF(\mathcal{I}), s) = O(1/\epsilon^{2p+2})F^p(Opt(\mathcal{I}), \frac{s}{(1+2\epsilon)(1+\epsilon)^4})$$

Hence the result follows for the L_p norm of flow time.

3.2 Stretch Analysis

By Lemma 1, it follows that $S^p(SETF(\mathcal{I}), 1+\epsilon) \leq S^p(MLF(\mathcal{I}), 1)$. Similarly, since each job in $J_i \in \mathcal{I}$ is at most $(1+\epsilon)^2$ times smaller than the corresponding job in $\mathcal{K}$, and also has size no more than that of the corresponding job in $\mathcal{J}$, we get that $S^p(MLF(\mathcal{I}), 1) \leq (1+\epsilon)^{2p} S^p(MLF(\mathcal{K}), 1)$. These together give us that

$$S^p(SETF(\mathcal{I}), 1+\epsilon) \leq (1+\epsilon)^{2p} S^p(MLF(\mathcal{K}), 1) \quad (8)$$

For a job of size $\epsilon[(1+\epsilon)^k - 1]$ in $\mathcal{K}$, the corresponding job of size $\epsilon^2(1+\epsilon)^k \in \mathcal{L}$ has equal flow time. Thus the ratio of the contributions to the pth power of stretch for $\mathcal{L}$ and $\mathcal{K}$ will be at least $(\frac{\epsilon^2(1+\epsilon)^k}{\epsilon[(1+\epsilon)^k-1]})^p$, which is at least $(\frac{\epsilon}{1-(1+\epsilon)^{-k}})^p$. Now since $\epsilon[(1+\epsilon)^k - 1] \geq 1$ for all valid job sizes in $\mathcal{K}$, we get that $(1+\epsilon)^{-k} \leq \frac{\epsilon}{1+\epsilon}$ and hence that $(\frac{\epsilon}{1-(1+\epsilon)^{-k}})^p \leq (\epsilon(1+\epsilon))^p$. Thus we get that $S^p(MLF(\mathcal{K}), 1) \leq (\epsilon(1+\epsilon))^p S^p(SJF(\mathcal{L}), 1)$. Now, by the $1+\epsilon$-speed, $O(1+1/\epsilon)$

competitiveness of SJF for stretch, it follows that $S^p(SJF(\mathcal{L}), 1+\epsilon) = O(1/\epsilon^p)S^p(Opt(\mathcal{L}), 1)$. Thus we get that

$$S^p(MLF(\mathcal{K}), 1) = O((1+\epsilon)^p)S^p(SJF(\mathcal{L}), 1) \tag{9}$$

We now prove a result similar to Lemma 3 for stretch.

Lemma 4 $S^p(Opt(\mathcal{L}), 1+\epsilon) = O(\frac{\log^{p+1}_{1+\epsilon} B}{\epsilon^p})S^p(SJF(\mathcal{J}), 1)$.

Proof: Set $x_i = \epsilon(1+\epsilon)^{-i}$ for $i = 1, \ldots, \log_{1+\epsilon}\log_{1+\epsilon}(P/\epsilon)$, and $x_i = \epsilon/\log_{1+\epsilon}(P/\epsilon)$ for $i > \log_{1+\epsilon}\log_{1+\epsilon}(P/\epsilon)$. We run the jobs in $\mathcal{L}(i)$ using SJF on a speed x_i processor. Notice that the total speedup required is $\sum_{i=1}^{\log_{1+\epsilon}(P/\epsilon)} x_i$ which is at most $1+\epsilon$.

By, lemma 2, $S^p(SJF(\mathcal{L}(i)), x_i)$ is at $(1/x_i^p)S^p(SJF(\mathcal{J}), 1)$. By simple algebraic calculation it can be seen that $\sum_i (\frac{1}{x_i})^p$ is $O(\frac{1}{\epsilon^p}\log^{p+1}_{1+\epsilon} P)$. □ Combining equations 8, 9, and lemma 4, we get that

$$S^p(SETF(\mathcal{I}), (1+\epsilon)^2) = O(\frac{(1+\epsilon)^{3p}}{\epsilon^p} \cdot \log^{p+1}_{1+\epsilon} P)S^p(SJF(\mathcal{J}), 1) \tag{10}$$

Thus we get that $S^p(MLF(\mathcal{K}), 1) \leq (\epsilon(1+\epsilon))^p S^p(SJF(\mathcal{L}), 1)$.

$$S^p(SJF(\mathcal{J}), 1+\epsilon) = O(1/\epsilon^p)S^p(Opt(\mathcal{J}), 1) \tag{11}$$

Also, since each job $J_i \in \mathcal{J}$ has size at most $(1+\epsilon)^2$ times more than the corresponding job $J_i \in \mathcal{I}$ (and is not smaller), we trivially have that

$$S^p(Opt(\mathcal{J}), (1+\epsilon)^2) \leq S^p(Opt(\mathcal{I}), 1) \tag{12}$$

Now combining equations 10, 11 and 12 it follows that $S^p(SETF, (1+\epsilon)^5, \mathcal{I}) = O(\frac{1}{\epsilon^{2p}} \cdot \log^{p+1}_{1+\epsilon} P)S^p(Opt(\mathcal{I}), 1)$, or equivalently that $S^p(SETF, (1+\epsilon)^5, \mathcal{I}) = O(\frac{1}{\epsilon^{3p+1}} \cdot \lg^{p+1} P)S^p(Opt(\mathcal{I}), 1)$. Thus the result for stretch follows.

4 Quasi-Polynomial Time Approximation Schemes

In this section, we describe our quasi-polynomial time approximation scheme for the minimizing the total flow time on a constant number of identical machines [4]. The idea can be extended to work for the case when the machines are unrelated and we consider weighted flow time (provided either P or W is polynomially bounded in n) [4]. Similarly the idea can be directly applied to give a QPTAS for the case of minimizing flow time with rejection costs [5].

4.1 Analysis

Let I be a problem instance with largest job size P. Without loss of generality we can assume that all release times are at most nP and that all jobs finish execution by time $2nP$ (otherwise we could reduce the problem into two disjoint problems). Let Opt denote the optimal schedule, we also abuse notation and use Opt to denote the total flow time under the optimal schedule.

Lemma 5 *Given I, rounding up the job sizes and release times to a multiple of $\epsilon P/n^2$ increases the cost of the optimum solution by a factor of at most $(1+2\epsilon)$.*

Proof: Given a schedule for the original instance, rounding up the job sizes adds at most $n \cdot \epsilon P/n^2 = \epsilon P/n$ to the flow time of each job. Similarly rounding up the release times adds at most $\epsilon P/n^2$ to the flow time of each job. Thus, the total flow time is affected by at most $2\epsilon P \leq 2\epsilon Opt$. □

Let I' denote this rounded instance. By Lemma 5 we can assume that all job sizes are integers in the range $[1, n^2/\epsilon]$. Similarly, as no release time in I is more than $2nP$, all release times in I' are integers in the range $[1, n^3\epsilon]$. We will obtain a schedule $S(I')$ with optimum total flow time for I'. Clearly, all events (arrivals and departures) under $S(I')$ at integral times in the range $[1, 2n^3/\epsilon]$. Also it directly seen that a schedule $S(I)$ for I follows from $S(I')$ and that the total flow time under $S(I)$ is at most that under $S(I')$. Henceforth, we only consider I'.

We first divide the jobs into $O(\frac{1}{\epsilon}\log n)$ classes. We say that a job with size p_j lies in class i, iff $(1+\epsilon)^{i-1} \leq p_j < (1+\epsilon)^i$. Consider the optimum algorithm. Let $S_j(i,t)$ denote the jobs of class i on machine j which are alive at time t. Note that the class of a job depends only on its initial processing time and hence does not change with time. It is easy to see that on each machine the jobs are processed in the order or shortest remaining processing time (SRPT). Thus we have that, for any $j = 1, \ldots, m$ and any time t, at most one job in $S_j(i,t)$ has a remaining processing time that is not in the range $(1+\epsilon)^{i-1}$ and $(1+\epsilon)^i$.

We now define the state of the algorithm $Q(t)$ at time t. For each class i and each machine j, we store at most $\frac{1}{\epsilon}+1$ numbers: the first $\frac{1}{\epsilon}$ are the remaining processing times of the $\frac{1}{\epsilon}$ jobs in $S_j(i,t)$ with the largest remaining processing time; the last entry is the sum of the remaining processing times of the rest of the jobs in $S_j(i,t)$. Notice that as each job has size at most n^2/ϵ, there are at most $(n^2/\epsilon)^{1/\epsilon} = O(n^{2/\epsilon})$ possible choices for the first $1/\epsilon$ entries and n^3/ϵ possible choices for the sum of remaining sizes. Finally, since there there are at most $O(\log n/\epsilon)$ classes and m machines, the total number of distinct states at any step is at most $n^{O(m\log n/\epsilon^2)}$.

The following lemma shows how this information helps us in estimating the number of unfinished jobs at any point of time.

Lemma 6 *We can estimate the number of jobs in queue at time t to within a factor of $(1+2\epsilon)$ using the information in $Q(t)$.*

Proof: If there are fewer than $1/\epsilon$ jobs in level i, we know their number precisely because we store their remaining processing times precisely. Let us now consider the case when there are more than $1/\epsilon$ jobs in some level i.

Suppose at first that the remaining processing times of all these jobs lies between $(1+\epsilon)^{i-1}$ and $(1+\epsilon)^i$. Then, by assuming that all the jobs have size $(1+\epsilon)^{i-1}$ and computing the number of jobs using the total remaining processing time, our estimate is off from the correct number by at most a factor of $1+\epsilon$.

Finally, as at most one job in every level i lies outside the range $(1+\epsilon)^{i-1}$ and $(1+\epsilon)^i$. Thus our estimate above could be off by another job. However, there are at least $1/\epsilon$ unfinished jobs. Thus we get an estimate within a factor of $1+2\epsilon$. □

Thus the algorithm is a large dynamic program with $O(n^3/\epsilon)$ times $n^{O(m \log n/\epsilon^2)}$ states, and for each state store the value of the minimum total flow time that can be achieved if that state is reached.

Now we only need to show how to update the state of the algorithm with time. When a new job arrives, we have m choices for the machine to which this job can be assigned. Once a machine is decided, the size of the job determines the class to which it belongs. Also, it is straightforward to update the state, as either the job is added to the first $1/\epsilon$ jobs in $S_j(i,t)$, or else if it is smaller that the $1/\epsilon$ largest jobs in its class, then its size is added to the $(1/\epsilon+1)^{th}$ entry. When the algorithm works on a job in level i on machine j,

Now consider the case when there are no arrivals. Suppose at time t, the algorithm works on the state $s(j)$ on machine j. We do not know $s(j)$, but we can try out all possible $O(\log n/\epsilon)^m$ choices for the different $s(j)$ on each machine j. For a fixed choice of $s(j)$, clearly the algorithm works on the job with the least remaining processing time in the class $s(j)$. So we either need to decrement the $\frac{1}{\epsilon}+1^{\text{th}}$ entry of $s(j)$ by one, or if there are no more than $\frac{1}{\epsilon}$ jobs, then the last non-zero entry is decremented by 1.

Thus we have shown that:

Theorem 7 *The above algorithm gives a $(1+\epsilon)$ approximation for the problem of minimizing total flow time on m identical machines and runs in time $n^{O(m \log n/\epsilon^2)}$.*

5 Other Open Problems

Some problems which do not fall in the framework that we discussed in the previous sections, but I would like to study further for the thesis are the following:

1. *Broadcast Scheduling without Resource Augmentation*

   In the broadcast scheduling model, there is a central server with k different files, each of size 1. Multiple users could request a file at different times. If the server broadcasts a file, it reaches all users *simultaneously*. The goal is to minimize the total flow time offline. The only results known use resource augmentation [26, 19, 20]. It is open if a non-trivial approximation ratio can be obtained without resource augmentation.

2. *$O(1)$-speed $O(1)$-approximation algorithm for non-preemptive total flow time*

   Throughout this proposal we only considered problems in a preemptive setting. It is known that no non-preemptive algorithm can obtain an approximation ratio better than $O(n^{1/2})$ for a single machine [27]. Given the strong lower bound, it is natural to study the problem in the resource augmentation model. An $O(\min(\log P, \log n))$-speed $O(1)$-approximation algorithm in known [33]. However it is open whether an $O(1)$-speed $O(1)$-approximation algorithm exists.

References

[1] A. Avidor, Y. Azar, and J. Sgall. Ancient and new algorithms for load balancing in the l_p norm. *Algorithmica*, 29(3):422–441, 2001.

[2] B. Awerbuch, Y. Azar, E. Grove, M. Kao, P. Krishnan, and J. Vitter. Load balancing in the l_p norm. In *IEEE Symposium on Foundations of Computer Science*, 1995.

[3] B. Awerbuch, Y. Azar, S. Leonardi, and O. Regev. Minimizing the flow time without migration. In *ACM Symposium on Theory of Computing*, pages 198–205, 1999.

[4] N. Bansal. Approximation schemes for flow time on multiple machines, Carnegie Mellon University, Technical Report, CMU-CS-03-103, Jan 2003.

[5] N. Bansal, A. Blum, S. Chawla, and K. Dhamdhere. Scheduling for flow time with admission control, manuscript, 2002.

[6] N. Bansal and K. Dhamdhere. Minimizing weighted flow time. In *Symposium on Discrete Algorithms SODA*, pages 508–516, 2003.

[7] N. Bansal, K. Dhamdhere, J. Konemann, and A. Sinha. Non-clairvoyant scheduling for minimizing mean slowdown. In *Symposium on Theoretical Aspects of Computer Science (STACS)*, to appear, 2003.

[8] N. Bansal and K. Pruhs. Server scheduling in the l_p norm: A rising tide lifts all boats. In *Symposium on Theory of Computing (STOC)*, to appear, 2003.

[9] Y. Bartal, S. Leonardi, A. Marchetti-Spaccamela, J. Sgall, and L. Stougie. Multiprocessor scheduling with rejection. In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 1996.

[10] L. Becchetti and S. Leonardi. Non-clairvoyant scheduling to minimize the average flow time on single and parallel machines. In *ACM Symposium on Theory of Computing (STOC)*, pages 94–103, 2001.

[11] L. Becchetti, S. Leonardi, A. M. Spaccamela, and K. Pruhs. Online weighted flow time and deadline scheduling. In *RANDOM-APPROX*, pages 36–47, 2001.

[12] M. Bender, S. Chakrabarti, and S. Muthukrishnan. Flow and stretch metrics for scheduling continuous job streams. In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 270–279, 1998.

[13] M. Bender, S. Muthukrishnan, and R. Rajaraman. Improved algorithms for stretch scheduling. In *13th Annual ACM-SIAM Symposium on Discrete Algorithms*, 2002.

[14] A. Borodin and R. El-Yaniv. *On-Line Computation and Competitive Analysis.* Cambridge University Press, 1998.

[15] C. Chekuri and S. Khanna. Approximation schemes for preemptive weighted flow time. In *ACM Symposium on Theory of Computing (STOC)*, 2002.

[16] C. Chekuri, S. Khanna, and A. Zhu. Algorithms for weighted flow time. In *ACM Symposium on Theory of Computing (STOC)*, 2001.

[17] D. Engels, D. Karger, S. Kolliopoulos, S. Sengupta, R. Uma, and J. Wein. Techniques for scheduling with rejection. *European Symposium on Algorithms*, pages 490–501, 1998.

[18] L. Epstein and J. Sgall. Approximation schemes for scheduling on uniformly related and identical parallel machines. In *European Symposium on Algorithms (ESA)*, pages 151–162, 1999.

[19] R. Gandhi, S. Khuller, Y. Kim, and Y-C. Wan. Algorithms for minimizing response time in broadcast scheduling. In *Integer Programming and Combinatorial Optimization (IPCO)*, pages 425–438, 2002.

[20] R. Gandhi, S. Khuller, S. Parthasarathy, and A. Srinivasan. Dependent rounding in bipartite graphs. In *Proc. IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 323–332, 2002.

[21] D. S. Hochbaum and D. B. Shmoys. Using dual approximation algorithms for scheduling problems theoretical and practical results. *Journal of the ACM (JACM)*, 34(1):144–162, 1987.

[22] E. Horowitz and S. Sahni. Exact and approximate algorithms for scheduling nonidentical processors. *Journal of the ACM (JACM)*, 23(2):317–327, 1976.

[23] K. Jansen and L. Porkolab. Improved approximation schemes for scheduling unrelated parallel machin es. In *ACM symposium on Theory of computing*, pages 408–417, 1999.

[24] B. Kalyanasundaram and K. Pruhs. Minimizing flow time nonclairvoyantly. In *IEEE Symposium on Foundations of Computer Science*, pages 345–352, 1997.

[25] B. Kalyanasundaram and K. Pruhs. Speed is as powerful as clairvoyance. *Journal of the ACM*, 47(4):617–643, 2000.

[26] B. Kalyanasundaram, K. Pruhs, and M. Velauthapillai. Scheduling broadcasts in wireless networks. In *European Symposium on Algoirthms, ESA*, pages 290–301, 2000.

[27] H. Kellerer, T. Tautenhahn, and G. J. Woeginger. Approximability and nonapproximability results for minimizing total flow time on a single machine. In *ACM Symposium on Theory of Computing (STOC)*, pages 418–426, 1996.

[28] J. Lenstra, A. Kan, and P. Brucker. Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1:343–362, 1977.

[29] J. Lenstra, D. Shmoys, and E. Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical Programming*, 46:259–271, 1990.

[30] S. Leonardi and D. Raz. Approximating total flow time on parallel machines. In *ACM Symposium on Theory of Computing (STOC)*, pages 110–119, 1997.

[31] R. Motwani, S. Phillips, and E. Torng. Nonclairvoyant scheduling. *Theoretical Computer Science*, 130(1):17–47, 1994.

[32] S. Muthukrishnan, R. Rajaraman, A. Shaheen, and J. Gehrke. Online scheduling to minimize average stretch. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 433–442, 1999.

[33] C. A. Phillips, C. Stein, E. Torng, and J. Wein. Optimal time-critical scheduling via resource augmentation. In *ACM Symposium on Theory of Computing (STOC)*, pages 140–149, 1997.

[34] P. Schuurman and G. J. Woeginger. Polynomial time approximation algorithms for machine scheduling: Ten open problems. *Journal of Scheduling*, 2:203 – 213, 1999.